Relazione del Progetto d'Esame:

Calcolatore del Punteggio di una Partita di Bowling

Daniele Tontini
Matricola: 328467

Tommaso Vincenzi
Matricola: 327812

Università degli Studi di Urbino Carlo Bo
Insegnamento di Programmazione Logica e Funzionale
Anno Accademico 2025/2026

1 Specifica del Problema

Scrivere un programma Haskell e un programma Prolog che acquisiscono da tastiera la sequenza dei birilli abbattuti in ciascun lancio di una partita di bowling e poi stampano su schermo il punteggio ottenuto in ciascuno dei dieci turni della partita, il punteggio totale raggiunto al termine di ciascun turno e il punteggio totale della partita.

2 Analisi del Problema

2.1 Dati di Ingresso del Problema

I dati di ingresso del problema sono rappresentati da una sequenza finita di numeri naturali, ciascuno dei quali esprime il numero di birilli abbattuti in un singolo lancio di una partita di bowling, elencati nell'ordine in cui i lanci sono stati effettuati.

2.2 Dati di Uscita del Problema

I dati di uscita del problema sono rappresentati:

- da dieci numeri naturali, uno per ciascun turno della partita, che ne esprimono il punteggio;
- da dieci numeri naturali che esprimono il punteggio totale raggiunto al termine di ciascun turno;
- da un ulteriore numero naturale che esprime il punteggio totale della partita.

2.3 Relazioni Intercorrenti tra i Dati del Problema

Le relazioni che legano i dati di ingresso ai dati di uscita coincidono con le regole di punteggio del gioco del bowling, delle quali si riporta di seguito la formalizzazione.

Struttura della partita e notazione

Una partita è composta da dieci turni. In ciascuno dei primi nove turni il giocatore effettua due lanci, a meno che non realizzi uno strike, nel qual caso il turno si chiude con un solo lancio; nel decimo turno il giocatore effettua tre lanci se realizza uno strike o uno spare, e due lanci altrimenti.

Sia $l_1, l_2, \ldots, l_n$ la sequenza dei lanci, dove l_i è il numero di birilli abbattuti dall'i-esimo lancio, e sia $f(t)$ l'indice del primo lancio del turno t, con $f(1) = 1$. Si indica con p_t il punteggio del turno t, con P_t il punteggio totale al termine del turno t e con P il punteggio totale della partita.

Vincoli sui singoli lanci

All'inizio di ogni turno i dieci birilli sono ricomposti nella loro disposizione completa, e i birilli abbattuti da un lancio non vengono ripristinati per il lancio successivo effettuato sulla stessa disposizione. Ogni lancio soddisfa quindi

$$0 \leq l_i \leq 10 \qquad \text{per ogni } i,$$

mentre i due lanci di uno stesso turno, quando il turno ne comprende due, non possono abbatterne complessivamente più di dieci:

$$l_{f(t)} < 10 \implies l_{f(t)} + l_{f(t)+1} \leq 10 \qquad \text{per } 1 \leq t \leq 9.$$

Se invece $l_{f(t)} = 10$ il turno si chiude con un solo lancio e $l_{f(t)+1}$ appartiene già al turno successivo, per cui i due valori non sono vincolati fra loro.

Due situazioni previste dalle regole del gioco non alterano tali vincoli e non incidono sul calcolo del punteggio, per cui non richiedono alcuna rappresentazione fra i dati del problema:

- lo *split*, ovvero una particolare disposizione dei birilli rimasti in piedi dopo il primo lancio, è una semplice annotazione sulla scheda di punteggio e lascia il punteggio invariato;
- il *fallo* e più in generale l'abbattimento irregolare comportano che al lancio sia attribuito un numero di birilli abbattuti pari a zero, il che rientra nel vincolo $0 \leq l_i \leq 10$ già enunciato.

Modo di chiusura di un turno e relativo punteggio

Per ogni turno t con $1 \leq t \leq 9$ si distinguono tre casi, mutuamente esclusivi ed esaustivi, che ne determinano il punteggio e l'indice del primo lancio del turno successivo:

- *Strike*: la disposizione completa dei birilli è abbattuta dal primo lancio del turno, ovvero $l_{f(t)} = 10$. Il turno si chiude con un solo lancio, quindi $f(t+1) = f(t)+1$, e il suo punteggio è dieci più il numero di birilli abbattuti dai due lanci successivi:

$$p_t = 10 + l_{f(t)+1} + l_{f(t)+2}.$$

- *Spare*: i birilli rimasti in piedi dopo il primo lancio sono abbattuti dal secondo lancio dello stesso turno, ovvero $l_{f(t)} < 10$ e $l_{f(t)} + l_{f(t)+1} = 10$. Il turno si chiude con due lanci, quindi $f(t+1) = f(t) + 2$, e il suo punteggio è dieci più il numero di birilli abbattuti dal lancio successivo:

$$p_t = 10 + l_{f(t)+2}.$$

- *Turno aperto*: i due lanci del turno non abbattono tutti e dieci i birilli, ovvero $l_{f(t)}+l_{f(t)+1} < 10$. Il turno si chiude con due lanci, quindi $f(t+1) = f(t) + 2$, e il suo punteggio è la semplice somma dei due lanci, senza alcun bonus:

$$p_t = l_{f(t)} + l_{f(t)+1}.$$

I due casi particolari dello strike che hanno un nome proprio non costituiscono regole aggiuntive, ma conseguenze immediate della formula precedente:

- *Doppio*: due strike consecutivi. Poiché il secondo dei due turni si chiude con un solo lancio, i due lanci successivi al primo strike sono il secondo strike stesso e il primo lancio del turno ancora seguente, per cui $p_t = 20 + l_{f(t)+2}$.
- *Triplo*: tre strike consecutivi, da cui $p_t = 30$. Ne discende che il punteggio massimo di una partita, pari a 300, si ottiene se e solo se il giocatore realizza dodici strike consecutivi.

Il decimo turno

Il decimo turno costituisce l'eccezione principale alle regole precedenti. Uno strike o uno spare realizzati in esso danno diritto ai lanci di bonus, i quali sono effettuati nel turno stesso anziché nei turni successivi, che non esistono. I birilli abbattuti da tali lanci sono semplicemente sommati e non generano a loro volta ulteriori bonus, per cui il punteggio del decimo turno è la somma dei lanci residui,

$$p_{10} = \sum_{i=f(10)}^{n} l_i,$$

dove la lunghezza della sequenza dipende dall'esito del turno:

- se $l_{f(10)} = 10$ il turno si chiude con uno strike e seguono due lanci di bonus, con $n = f(10) + 2$. I birilli sono ricomposti dopo lo strike, per cui il vincolo sulla somma dei due lanci successivi non si applica al primo bonus; se anche $l_{f(10)+1} = 10$ i birilli sono nuovamente ricomposti e $0 \leq l_{f(10)+2} \leq 10$, altrimenti i due bonus insistono sulla stessa disposizione e $l_{f(10)+1} + l_{f(10)+2} \leq 10$;
- se $l_{f(10)} < 10$ e $l_{f(10)} + l_{f(10)+1} = 10$ il turno si chiude con uno spare e segue un solo lancio di bonus, effettuato su una disposizione ricomposta e quindi tale che $0 \leq l_{f(10)+2} \leq 10$, con $n = f(10) + 2$;
- se $l_{f(10)} + l_{f(10)+1} < 10$ il turno è aperto, non dà diritto ad alcun bonus e $n = f(10) + 1$.

Il decimo turno interviene inoltre nel calcolo del punteggio del nono: se il nono turno si chiude con uno strike, i due lanci di bonus che gli competono sono i primi due lanci del decimo turno, che possono a loro volta essere uno strike seguito da un lancio di bonus; se si chiude con uno spare, il bonus è il primo lancio del decimo turno.

Duplice ruolo dei lanci di bonus

Una proprietà essenziale, che discende dalle definizioni di strike e di spare e che governa l'intero calcolo, è che i lanci utilizzati come bonus non vengono sottratti al turno cui appartengono: essi concorrono sia al punteggio del turno che li usa come bonus, sia al punteggio del proprio turno. Un medesimo lancio può quindi essere conteggiato fino a tre volte, come accade al primo lancio che segue un triplo. Non si tratta pertanto di una ripartizione dei lanci fra i turni, ma di una sovrapposizione controllata.

Ne consegue che il punteggio di un turno può dipendere non soltanto dai lanci del turno stesso, ma anche da quelli che lo seguono: dai due lanci successivi se il turno si chiude con uno strike, dal solo lancio successivo se si chiude con uno spare, da nessuno se il turno è aperto. La dipendenza è dunque orientata in avanti e limitata a due lanci; il decimo turno, non essendo seguito da alcun turno, non influenza il punteggio di alcun altro turno, il che rende la dipendenza aciclica e ne garantisce la buona definizione.

Punteggi cumulativi e limiti quantitativi

Dalle definizioni precedenti discendono le seguenti proprietà, utili ai fini della risoluzione del problema:

- Il punteggio totale al termine del turno t e il punteggio totale della partita sono dati rispettivamente da
$$P_t = \sum_{k=1}^{t} p_k \qquad \text{e} \qquad P = P_{10} = \sum_{t=1}^{10} p_t,$$
e la successione $P_1, P_2, \ldots, P_{10}$ è monotona non decrescente in quanto $p_t \geq 0$ per ogni t.
- Vale $0 \leq p_t \leq 30$ per ogni t e quindi $0 \leq P \leq 300$. Il punteggio massimo $P = 300$ si ottiene se e solo se ogni lancio è uno strike, mentre $P = 0$ si ottiene se e solo se nessun lancio abbatte birilli.
- La lunghezza della sequenza dei lanci non è nota a priori e soddisfa $11 \leq n \leq 21$: il minimo si ha quando i primi nove turni si chiudono tutti con uno strike e il decimo turno è aperto, il massimo quando nessun turno si chiude con uno strike e il decimo dà diritto a un bonus.
- Non tutte le sequenze finite di numeri naturali descrivono una partita: esse devono soddisfare simultaneamente i vincoli sui singoli lanci, i vincoli sulle coppie di lanci effettuati sulla stessa disposizione di birilli, la suddivisione in esattamente dieci turni e l'esaurirsi della sequenza con l'ultimo lancio del decimo turno, senza lanci mancanti né in eccesso.

Esempio di calcolo

Si riporta una partita di esempio che esercita tutte le regole enunciate. La notazione convenzionale della scheda di punteggio indica con `X` uno strike, con `/` uno spare, con `-` un lancio che non abbatte alcun birillo e con `F` un fallo; l'uno e l'altro valgono zero birilli abbattuti.

Turno	Notazione	Calcolo del punteggio	p_t	P_t
1	`X`	$10 + 10 + 10$ (triplo)	30	30
2	`X`	$10 + 10 + 7$ (doppio)	27	57
3	`X`	$10 + 7 + 2$	19	76
4	`7 2`	$7 + 2$	9	85
5	`8 /`	$10 + 0$ (il lancio di bonus è un fallo)	10	95
6	`F 9`	$0 + 9$	9	104
7	`X`	$10 + 7 + 3$	20	124
8	`7 /`	$10 + 9$	19	143
9	`9 -`	$9 + 0$	9	152
10	`X X 8`	$10 + 10 + 8$	28	180

La corrispondente sequenza dei lanci è

10, 10, 10, 7, 2, 8, 2, 0, 9, 10, 7, 3, 9, 0, 10, 10, 8,

composta da diciassette lanci, e il punteggio totale della partita è 180. Si osservi come il quinto turno, chiuso con uno spare, riceva un bonus nullo perché il lancio successivo è un fallo, e come il decimo turno sommi i propri tre lanci senza generare alcun bonus ulteriore.

3 Progettazione dell'Algoritmo

3.1 Scelte di Progetto

La sequenza dei lanci si presta in modo naturale a essere rappresentata mediante una struttura dati lineare. Questa verrà allocata dinamicamente in quanto la lunghezza della sequenza non è fissa, pur essendo compresa fra undici e ventuno, in maniera da non imporre limitazioni rispetto alla specifica del problema.

Ciascun turno verrà rappresentato mediante un valore composto che ne registra sia il modo di chiusura, ovvero strike, spare oppure turno aperto, sia il punteggio già comprensivo degli eventuali bonus. Questa scelta consente di stampare un prospetto descrittivo della partita senza dover riesaminare la sequenza dei lanci, e rende esplicita nel dato la classificazione introdotta nell'analisi del problema.

I dieci turni verranno determinati mediante un'unica scansione della sequenza dei lanci da sinistra a destra, portando come parametro il numero del turno corrente, necessario perché il decimo turno segue regole proprie. La porzione di sequenza non ancora consumata contiene esattamente i lanci che fungono da bonus per il turno corrente: tali lanci verranno pertanto soltanto letti, senza essere consumati, e resteranno disponibili per i turni successivi che li consumeranno a loro volta. Si evita in questo modo sia una seconda scansione della sequenza sia la duplicazione dei lanci in strutture ausiliarie, sfruttando il fatto che la dipendenza fra i turni è orientata in avanti e limitata a due lanci.

I punteggi totali al termine di ciascun turno verranno calcolati con un'unica scansione dei punteggi dei turni per mezzo di un accumulatore, anziché ricalcolare ogni volta una somma parziale.

Il prospetto dei turni verrà infine presentato incolonnando i valori numerici, in modo da renderne immediata la lettura e il confronto.

3.2 Passi dell'Algoritmo

I passi dell'algoritmo per risolvere il problema sono i seguenti:

- Acquisire la sequenza dei lanci, richiedendone nuovamente l'immissione qualora l'input non sia una sequenza di numeri naturali oppure non descriva una partita valida secondo le relazioni evidenziate nell'analisi del problema.
- Determinare i dieci turni della partita, a partire dal primo turno e dall'intera sequenza dei lanci:
  - Caso base: il turno da determinare è il decimo, per il quale la sequenza residua deve esaurirsi e il punteggio è la somma dei lanci residui:
    - se il primo lancio residuo abbatte tutti i birilli e sono residui esattamente altri due lanci ammissibili come bonus, il turno si chiude con uno strike;
    - altrimenti, se i primi due lanci residui abbattono tutti i birilli ed è residuo esattamente un ulteriore lancio ammissibile come bonus, il turno si chiude con uno spare;
    - altrimenti, se sono residui esattamente due lanci che non abbattono tutti i birilli, il turno è aperto.
  - Caso generale con strike: il primo lancio non ancora consumato abbatte tutti i birilli; il punteggio del turno è dieci più la somma dei due lanci che lo seguono nella sequenza, i quali non vengono consumati, e si procede ricorsivamente col turno successivo e col resto della sequenza privato del solo lancio consumato.
  - Caso generale con spare: i primi due lanci non ancora consumati formano una coppia ammissibile, il primo non abbatte tutti i birilli e insieme li abbattono tutti; il punteggio

del turno è dieci più il lancio che li segue, il quale non viene consumato, e si procede ricorsivamente col turno successivo e col resto della sequenza privato dei due lanci consumati.
  - Caso generale con turno aperto: i primi due lanci non ancora consumati formano una coppia ammissibile e non abbattono tutti i birilli; il punteggio del turno è la loro somma e si procede ricorsivamente come nel caso precedente.
  - Caso di errore: nessuno dei casi precedenti è applicabile, per cui la sequenza non descrive una partita valida. Questo passo realizza dunque anche la validazione richiesta al passo di acquisizione, senza che le regole del gioco debbano essere enunciate una seconda volta in forma di controlli separati.

- Calcolare i punteggi totali al termine di ciascun turno, a partire da un totale accumulato inizialmente nullo:
  - Caso base: se non restano turni da considerare, la sequenza dei totali è vuota.
  - Caso generale: se resta almeno un turno, il totale al termine di tale turno è la somma del totale accumulato e del punteggio del turno, e si procede ricorsivamente coi turni rimanenti a partire da tale totale.

- Calcolare il punteggio totale della partita:
  - Caso base: se non restano turni da considerare, il punteggio totale è nullo.
  - Caso generale: se resta almeno un turno, il punteggio totale è la somma del punteggio di tale turno e del punteggio totale dei turni rimanenti.

- Stampare il prospetto della partita:
  - Caso base: se non restano turni da presentare, non si stampa nulla.
  - Caso generale: se resta almeno un turno, se ne stampa il numero d'ordine, il punteggio, il totale al termine del turno e il modo di chiusura, e si procede ricorsivamente coi turni rimanenti.

- Stampare il punteggio totale della partita.

Il calcolo del punteggio di un turno chiuso con uno strike o con uno spare si avvale di un sottoprogramma che somma i primi lanci di una sequenza senza consumarli, il quale realizza la lettura in avanti descritta nelle scelte di progetto. La verifica di ammissibilità dei lanci si avvale di tre ulteriori sottoprogrammi, che stabiliscono rispettivamente se un singolo lancio abbatte un numero ammissibile di birilli, se due lanci effettuati sulla stessa disposizione sono compatibili fra loro e se i due lanci di bonus del decimo turno sono ammissibili. L'incolonnamento dei valori numerici si avvale infine di un sottoprogramma che allinea a destra un numero entro una larghezza prefissata.

4 Implementazione dell'Algoritmo

Le differenze implementative significative tra il programma Haskell e il programma Prolog sono le seguenti:

- *Esito della determinazione dei turni*: nel programma Haskell la funzione `turni` restituisce un valore di tipo `Maybe [Turno]`, con un caso esplicito che produce `Nothing` quando nessuna regola è applicabile, e la funzione ausiliaria `aggiungi` propaga tale esito negativo lungo tutta la ricorsione; nel programma Prolog il predicato `turni` non ha alcuna clausola corrispondente al caso di errore e l'esito negativo è ottenuto per fallimento del predicato, gestito dal predicato `controlla_partita` mediante cut. Il passo dell'algoritmo "nessuno dei casi precedenti è applicabile" è quindi realizzato esplicitamente in un programma e implicitamente nell'altro, e per la stessa ragione la funzione `aggiungi` non ha alcun corrispettivo in Prolog, dove la lista dei turni viene costruita direttamente nella testa della clausola per unificazione.
- *Rappresentazione del turno*: nel programma Haskell il turno è un valore del tipo di dato algebrico `Turno`, i cui tre costruttori `Strike`, `Spare` e `Aperto` sono discriminati mediante pattern matching; nel programma Prolog il turno è un termine composto della forma `strike(P)`, `spare(P)` oppure `aperto(P)`, discriminato per unificazione.
- *Validazione dell'input*: nel programma Haskell l'input viene analizzato mediante la funzione `readMaybe`, che restituisce `Nothing` anziché terminare l'esecuzione in caso di errore, e il controllo che i lanci siano numeri interi è quindi svolto una sola volta in fase di analisi; nel programma Prolog il termine letto da `read` può essere di qualunque forma, per cui la natura intera dei lanci è verificata dal predicato `lancio_valido` mediante il predicato predefinito `integer`.
- *Costruzione del prospetto*: nel programma Haskell la formattazione è interamente pura, in quanto le funzioni `righe` e `riga` costruiscono le righe di testo e l'azione di input/output `stampa_righe` si limita a stamparle, mantenendo netta la separazione fra la parte pura e la parte impura, espressa nel tipo `IO`; nel programma Prolog, dove tale distinzione non sussiste, il predicato `stampa_turni` costruisce e stampa ciascuna riga contestualmente. Ne consegue che in Haskell la funzione `allinea` opera su stringhe, anteponendo spazi fino alla larghezza voluta, mentre in Prolog il predicato omonimo determina aritmeticamente il numero di cifre del valore e stampa altrettanti spazi.
- *Selezione dei casi*: nel programma Haskell i casi generali che condividono lo stesso pattern sono discriminati mediante guardie esaustive, l'ultima delle quali è `otherwise`, mentre le equazioni di `turno_finale` sfruttano la ricaduta sull'equazione successiva quando la guardia non è soddisfatta; nel programma Prolog le clausole corrispondenti hanno condizioni mutuamente esclusive, per cui non è necessario alcun cut per selezionare il caso corretto e l'unico cut del programma è quello che rende deterministica l'acquisizione dell'input.

File sorgente `punteggio_bowling.hs`:

```
{- Programma Haskell per calcolare il punteggio di una partita di
   bowling. -}

import Text.Read -- necessario per usare readMaybe, che analizza l'input
                 -- restituendo Nothing anziche' terminare in caso di errore

{- Il tipo di dato Turno rappresenta il risultato di uno dei dieci turni
   di una partita di bowling:
   - il costruttore Strike rappresenta un turno chiuso abbattendo tutti
     i birilli con il primo lancio;
   - il costruttore Spare rappresenta un turno chiuso abbattendo tutti i
     birilli con i due lanci del turno;
```

```
   - il costruttore Aperto rappresenta un turno i cui due lanci non
     abbattono tutti i birilli,
   dove l'unico argomento di ciascun costruttore e' il punteggio del
   turno, gia' comprensivo degli eventuali bonus. -}
data Turno = Strike Int | Spare Int | Aperto Int

main :: IO ()
main = do partita <- acquisisci_partita
          putStrLn ""
          putStrLn ">> Punteggio dei singoli turni:"
          stampa_righe (righe 1 partita (totali 0 partita))
          putStrLn ">> Punteggio totale della partita:"
          putStrLn $ show (punteggio_totale partita)

{- L'azione di input/output acquisisci_partita acquisisce la sequenza
   dei lanci di una partita, delegando ad analizza_partita il calcolo
   dei dieci turni e a controlla_partita la verifica dell'esito
   dell'analisi dell'input. -}
acquisisci_partita :: IO [Turno]
acquisisci_partita = do putStr "Digita la sequenza dei lanci"
                        putStrLn " racchiusa tra parentesi quadre:"
                        lanci' <- getLine
                        controlla_partita
                          (analizza_partita (readMaybe lanci'))

{- L'azione parametrica di input/output controlla_partita restituisce i
   dieci turni calcolati se l'analisi dell'input e' riuscita, altrimenti
   segnala l'errore e richiede nuovamente la sequenza dei lanci:
   - il suo unico argomento e' l'esito dell'analisi dell'input. -}
controlla_partita :: Maybe [Turno] -> IO [Turno]
controlla_partita (Just partita) = return partita
controlla_partita Nothing        = do putStrLn ">> Partita non valida."
                                      acquisisci_partita

{- L'azione parametrica di input/output stampa_righe stampa una alla
   volta le righe del prospetto dei turni:
   - il suo unico argomento e' la lista delle righe da stampare. -}
stampa_righe :: [String] -> IO ()
stampa_righe []              = return ()
stampa_righe (riga' : righe') = do putStrLn riga'
                                   stampa_righe righe'

{- La funzione analizza_partita calcola i dieci turni della partita a
   partire dall'esito dell'analisi dell'input, restituendo Nothing se
   l'input non e' una lista di numeri interi oppure se la sequenza dei
   lanci non descrive una partita valida:
   - il suo unico argomento e' l'esito dell'analisi dell'input. -}
analizza_partita :: Maybe [Int] -> Maybe [Turno]
analizza_partita Nothing      = Nothing
analizza_partita (Just lanci) = turni 1 lanci

{- La funzione turni consuma la sequenza dei lanci e costruisce la lista
   dei turni della partita, restituendo Nothing se la sequenza non
   descrive una partita valida:
   - il primo argomento e' il numero del turno da calcolare;
   - il secondo argomento e' la sequenza dei lanci non ancora consumati.
   Caso base: il turno da calcolare e' il decimo, che segue regole
   proprie ed e' delegato a turno_finale.
   Caso generale con strike: il primo lancio abbatte tutti i birilli, il
   turno si chiude consumando un solo lancio e il suo punteggio e' dieci
   piu' la somma dei due lanci successivi, che vengono letti nella
   sequenza non ancora consumata senza essere consumati.
   Caso generale con spare: i due lanci del turno abbattono tutti i
```

```
   birilli e il punteggio e' dieci piu' il lancio successivo.
   Caso generale con turno aperto: i due lanci del turno non abbattono
   tutti i birilli e il punteggio e' la loro somma.
   Caso degenere: la sequenza dei lanci e' esaurita prima del decimo
   turno oppure i due lanci del turno non sono ammissibili. -}
turni :: Int -> [Int] -> Maybe [Turno]
turni 10 lanci       = turno_finale lanci
turni n (10 : resto) = aggiungi (Strike (10 + somma_primi 2 resto))
                                (turni (n + 1) resto)
turni n (a : b : resto)
  | not (coppia_valida a b) = Nothing
  | a + b == 10             = aggiungi (Spare (10 + somma_primi 1 resto))
                                       (turni (n + 1) resto)
  | otherwise               = aggiungi (Aperto (a + b))
                                       (turni (n + 1) resto)
turni _ _            = Nothing

{- La funzione turno_finale calcola il decimo turno, nel quale uno
   strike o uno spare danno diritto ai lanci di bonus e il punteggio e'
   la somma dei birilli abbattuti, senza ulteriori bonus:
   - il suo unico argomento e' la sequenza dei lanci non ancora
     consumati, che deve esaurirsi con il decimo turno.
   Caso con strike: il primo lancio abbatte tutti i birilli e seguono
   due lanci di bonus.
   Caso con spare: i primi due lanci abbattono tutti i birilli e segue
   un solo lancio di bonus.
   Caso con turno aperto: i due lanci non abbattono tutti i birilli e
   non danno diritto ad alcun bonus.
   Caso degenere: il numero dei lanci residui non e' coerente con
   l'esito del decimo turno. -}
turno_finale :: [Int] -> Maybe [Turno]
turno_finale [10, b, c]
  | bonus_validi b c                = Just [Strike (10 + b + c)]
turno_finale [a, b, c]
  | coppia_valida a b && a < 10 &&
    a + b == 10 && lancio_valido c  = Just [Spare (10 + c)]
turno_finale [a, b]
  | coppia_valida a b && a + b < 10 = Just [Aperto (a + b)]
turno_finale _                      = Nothing

{- La funzione aggiungi antepone un turno alla lista dei turni gia'
   calcolati, propagando l'eventuale esito negativo dell'analisi dei
   turni successivi:
   - il primo argomento e' il turno da anteporre;
   - il secondo argomento e' l'esito dell'analisi dei turni
     successivi. -}
aggiungi :: Turno -> Maybe [Turno] -> Maybe [Turno]
aggiungi _ Nothing      = Nothing
aggiungi t (Just altri) = Just (t : altri)

{- La funzione somma_primi calcola la somma dei primi lanci di una
   sequenza ed e' lo strumento con cui i turni chiusi con strike o con
   spare leggono in avanti i lanci successivi:
   - il primo argomento e' il numero di lanci da sommare;
   - il secondo argomento e' la sequenza dei lanci.
   Caso base: non resta alcun lancio da sommare.
   Caso degenere: la sequenza e' esaurita prima del previsto, situazione
   che si presenta soltanto in una partita incompleta e che viene poi
   respinta dal calcolo del decimo turno.
   Caso generale: il primo lancio contribuisce alla somma dei
   rimanenti. -}
somma_primi :: Int -> [Int] -> Int
somma_primi 0 _           = 0
```

```
somma_primi _ []          = 0
somma_primi n (a : resto) = a + somma_primi (n - 1) resto

{- La funzione bonus_validi verifica che i due lanci di bonus del decimo
   turno siano ammissibili:
   - il primo argomento e' il primo lancio di bonus;
   - il secondo argomento e' il secondo lancio di bonus.
   Caso base: il primo bonus abbatte tutti i birilli, che vengono percio'
   ricomposti, e il secondo bonus e' libero di assumere qualsiasi valore.
   Caso generale: i due bonus insistono sulla stessa disposizione di
   birilli e devono percio' formare una coppia ammissibile. -}
bonus_validi :: Int -> Int -> Bool
bonus_validi 10 c = lancio_valido c
bonus_validi b c  = coppia_valida b c

{- La funzione coppia_valida verifica che due lanci effettuati sulla
   stessa disposizione di birilli siano ammissibili, ovvero che non ne
   abbattano complessivamente piu' di dieci:
   - il primo argomento e' il primo dei due lanci;
   - il secondo argomento e' il secondo dei due lanci. -}
coppia_valida :: Int -> Int -> Bool
coppia_valida a b = lancio_valido a && lancio_valido b && a + b <= 10

{- La funzione lancio_valido verifica che un lancio abbatta un numero di
   birilli compreso tra zero e dieci:
   - il suo unico argomento e' il lancio da verificare. -}
lancio_valido :: Int -> Bool
lancio_valido a = a >= 0 && a <= 10

{- La funzione righe costruisce il prospetto dei turni sotto forma di
   lista di righe di testo:
   - il primo argomento e' il numero del primo turno da presentare;
   - il secondo argomento e' la lista dei turni;
   - il terzo argomento e' la lista dei corrispondenti punteggi
     cumulativi. -}
righe :: Int -> [Turno] -> [Int] -> [String]
righe _ []          _              = []
righe _ _           []             = []
righe n (t : altri) (c : restanti) = riga n t c
                                     : righe (n + 1) altri restanti

{- La funzione riga costruisce la riga di testo relativa a un singolo
   turno, riportandone il numero, il punteggio, il totale parziale della
   partita e il modo di chiusura del turno:
   - il primo argomento e' il numero del turno;
   - il secondo argomento e' il turno da presentare;
   - il terzo argomento e' il punteggio cumulativo al termine del
     turno. -}
riga :: Int -> Turno -> Int -> String
riga n t c = "Turno " ++ allinea 2 (show n) ++
             ":  punteggio " ++ allinea 2 (show (punteggio t)) ++
             "   totale parziale " ++ allinea 3 (show c) ++
             "   (" ++ nome_turno t ++ ")"

{- La funzione allinea allinea a destra una stringa entro una larghezza
   prefissata, anteponendo gli spazi necessari, in modo che le colonne
   del prospetto risultino incolonnate:
   - il primo argomento e' la larghezza desiderata;
   - il secondo argomento e' la stringa da allineare. -}
allinea :: Int -> String -> String
allinea l s | length s >= l = s
            | otherwise     = allinea l (' ' : s)
```

```
{- La funzione nome_turno restituisce la descrizione testuale del modo
   di chiusura di un turno:
   - il suo unico argomento e' il turno da descrivere. -}
nome_turno :: Turno -> String
nome_turno (Strike _) = "strike"
nome_turno (Spare  _) = "spare"
nome_turno (Aperto _) = "turno aperto"

{- La funzione totali calcola la lista dei punteggi cumulativi, ovvero
   il totale della partita al termine di ciascun turno:
   - il primo argomento e' il totale accumulato nei turni precedenti;
   - il secondo argomento e' la lista dei turni ancora da considerare. -}
totali :: Int -> [Turno] -> [Int]
totali _   []          = []
totali acc (t : altri) = totale : totali totale altri
                         where totale = acc + punteggio t

{- La funzione punteggio_totale calcola il punteggio complessivo della
   partita sommando i punteggi dei singoli turni:
   - il suo unico argomento e' la lista dei turni della partita. -}
punteggio_totale :: [Turno] -> Int
punteggio_totale []          = 0
punteggio_totale (t : altri) = punteggio t + punteggio_totale altri

{- La funzione punteggio restituisce il punteggio di un turno,
   indipendentemente dal modo di chiusura del turno:
   - il suo unico argomento e' il turno di cui calcolare il
     punteggio. -}
punteggio :: Turno -> Int
punteggio (Strike p) = p
punteggio (Spare  p) = p
punteggio (Aperto p) = p
```

File sorgente `punteggio_bowling.pl`:

```
/* Programma Prolog per calcolare il punteggio di una partita di
   bowling. */

/* Il risultato di uno dei dieci turni di una partita di bowling e'
   rappresentato mediante:
   - il termine strike(P), che rappresenta un turno chiuso abbattendo
     tutti i birilli con il primo lancio;
   - il termine spare(P), che rappresenta un turno chiuso abbattendo
     tutti i birilli con i due lanci del turno;
   - il termine aperto(P), che rappresenta un turno i cui due lanci non
     abbattono tutti i birilli,
   dove l'argomento P e' il punteggio del turno, gia' comprensivo degli
   eventuali bonus. */

main :- acquisisci_partita(Partita),
        nl,
        write('>> Punteggio dei singoli turni:'), nl,
        totali(0, Partita, Cumulativi),
        stampa_turni(1, Partita, Cumulativi),
        write('>> Punteggio totale della partita:'), nl,
        punteggio_totale(Partita, Totale),
        write(Totale), nl.

/* Il predicato acquisisci_partita acquisisce la sequenza dei lanci di
   una partita, delegando a controlla_partita la verifica della
   validita' dell'input:
   - il suo unico argomento e' la lista dei dieci turni calcolati. */
acquisisci_partita(Partita) :-
    write('Digita la sequenza dei lanci racchiusa tra parentesi'),
    write(' quadre e terminata da punto:'), nl,
    read(Lanci),
    controlla_partita(Lanci, Partita).

/* Il predicato controlla_partita restituisce i dieci turni calcolati se
   la sequenza acquisita descrive una partita valida, altrimenti segnala
   l'errore e richiede nuovamente la sequenza dei lanci:
   - il primo argomento e' la sequenza dei lanci acquisita;
   - il secondo argomento e' la lista dei dieci turni calcolati. */
controlla_partita(Lanci, Partita) :- turni(1, Lanci, Partita), !.
controlla_partita(_, Partita) :- write('>> Partita non valida.'), nl,
                                 acquisisci_partita(Partita).

/* Il predicato turni consuma la sequenza dei lanci e costruisce la
   lista dei turni della partita, fallendo se la sequenza non descrive
   una partita valida:
   - il primo argomento e' il numero del turno da calcolare;
   - il secondo argomento e' la sequenza dei lanci non ancora consumati;
   - il terzo argomento e' la lista dei turni calcolati.
   Caso base: il turno da calcolare e' il decimo, che segue regole
   proprie ed e' delegato a turno_finale.
   Caso generale con strike: il primo lancio abbatte tutti i birilli, il
   turno si chiude consumando un solo lancio e il suo punteggio e' dieci
   piu' la somma dei due lanci successivi, che vengono letti nella
   sequenza non ancora consumata senza essere consumati.
   Caso generale con spare: i due lanci del turno abbattono tutti i
   birilli e il punteggio e' dieci piu' il lancio successivo.
   Caso generale con turno aperto: i due lanci del turno non abbattono
   tutti i birilli e il punteggio e' la loro somma.
   Le condizioni delle tre clausole generali sono mutuamente esclusive,
   percio' non e' necessario alcun cut per selezionare il caso. */
turni(10, Lanci, [Turno]) :- turno_finale(Lanci, Turno).
turni(N, [10 | Resto], [strike(P) | Altri]) :-
```

```
    N < 10,
    somma_primi(2, Resto, S), P is 10 + S,
    N1 is N + 1, turni(N1, Resto, Altri).
turni(N, [A, B | Resto], [spare(P) | Altri]) :-
    N < 10, coppia_valida(A, B), A < 10, A + B =:= 10,
    somma_primi(1, Resto, S), P is 10 + S,
    N1 is N + 1, turni(N1, Resto, Altri).
turni(N, [A, B | Resto], [aperto(P) | Altri]) :-
    N < 10, coppia_valida(A, B), A + B < 10, P is A + B,
    N1 is N + 1, turni(N1, Resto, Altri).

/* Il predicato turno_finale calcola il decimo turno, nel quale uno
   strike o uno spare danno diritto ai lanci di bonus e il punteggio e'
   la somma dei birilli abbattuti, senza ulteriori bonus:
   - il primo argomento e' la sequenza dei lanci non ancora consumati,
     che deve esaurirsi con il decimo turno;
   - il secondo argomento e' il turno calcolato.
   Caso con strike: il primo lancio abbatte tutti i birilli e seguono
   due lanci di bonus.
   Caso con spare: i primi due lanci abbattono tutti i birilli e segue
   un solo lancio di bonus.
   Caso con turno aperto: i due lanci non abbattono tutti i birilli e
   non danno diritto ad alcun bonus. */
turno_finale([10, B, C], strike(P)) :-
    bonus_validi(B, C), P is 10 + B + C.
turno_finale([A, B, C], spare(P)) :-
    coppia_valida(A, B), A < 10, A + B =:= 10, lancio_valido(C),
    P is 10 + C.
turno_finale([A, B], aperto(P)) :-
    coppia_valida(A, B), A + B < 10, P is A + B.

/* Il predicato somma_primi calcola la somma dei primi lanci di una
   sequenza ed e' lo strumento con cui i turni chiusi con strike o con
   spare leggono in avanti i lanci successivi:
   - il primo argomento e' il numero di lanci da sommare;
   - il secondo argomento e' la sequenza dei lanci;
   - il terzo argomento e' la somma calcolata.
   Caso base: non resta alcun lancio da sommare.
   Caso degenere: la sequenza e' esaurita prima del previsto, situazione
   che si presenta soltanto in una partita incompleta e che viene poi
   respinta dal calcolo del decimo turno.
   Caso generale: il primo lancio contribuisce alla somma dei
   rimanenti. */
somma_primi(0, _, 0).
somma_primi(N, [], 0) :- N > 0.
somma_primi(N, [A | Resto], S) :-
    N > 0, N1 is N - 1, somma_primi(N1, Resto, S1), S is A + S1.

/* Il predicato bonus_validi verifica che i due lanci di bonus del
   decimo turno siano ammissibili:
   - il primo argomento e' il primo lancio di bonus;
   - il secondo argomento e' il secondo lancio di bonus.
   Caso base: il primo bonus abbatte tutti i birilli, che vengono percio'
   ricomposti, e il secondo bonus e' libero di assumere qualsiasi valore.
   Caso generale: i due bonus insistono sulla stessa disposizione di
   birilli e devono percio' formare una coppia ammissibile. */
bonus_validi(10, C) :- lancio_valido(C).
bonus_validi(B, C) :- coppia_valida(B, C), B < 10.

/* Il predicato coppia_valida verifica che due lanci effettuati sulla
   stessa disposizione di birilli siano ammissibili, ovvero che non ne
   abbattano complessivamente piu' di dieci:
   - il primo argomento e' il primo dei due lanci;
```

```
   - il secondo argomento e' il secondo dei due lanci. */
coppia_valida(A, B) :- lancio_valido(A), lancio_valido(B), A + B =< 10.

/* Il predicato lancio_valido verifica che un lancio abbatta un numero
   di birilli compreso tra zero e dieci:
   - il suo unico argomento e' il lancio da verificare. */
lancio_valido(A) :- integer(A), A >= 0, A =< 10.

/* Il predicato totali calcola la lista dei punteggi cumulativi, ovvero
   il totale della partita al termine di ciascun turno:
   - il primo argomento e' il totale accumulato nei turni precedenti;
   - il secondo argomento e' la lista dei turni ancora da considerare;
   - il terzo argomento e' la lista dei punteggi cumulativi. */
totali(_, [], []).
totali(Acc, [Turno | Altri], [Totale | Cumulativi]) :-
    punteggio(Turno, P),
    Totale is Acc + P,
    totali(Totale, Altri, Cumulativi).

/* Il predicato stampa_turni stampa il prospetto dei turni, riportando
   per ciascuno di essi il numero, il punteggio, il totale parziale
   della partita e il modo di chiusura del turno:
   - il primo argomento e' il numero del turno da stampare;
   - il secondo argomento e' la lista dei turni ancora da stampare;
   - il terzo argomento e' la lista dei corrispondenti punteggi
     cumulativi. */
stampa_turni(_, [], []).
stampa_turni(N, [Turno | Altri], [C | Restanti]) :-
    write('Turno '), allinea(2, N),
    punteggio(Turno, P),
    write(':  punteggio '), allinea(2, P),
    write('   totale parziale '), allinea(3, C),
    nome_turno(Turno, Nome),
    write('   ('), write(Nome), write(')'), nl,
    N1 is N + 1,
    stampa_turni(N1, Altri, Restanti).

/* Il predicato allinea stampa un numero allineato a destra entro una
   larghezza prefissata, anteponendo gli spazi necessari, in modo che le
   colonne del prospetto risultino incolonnate:
   - il primo argomento e' la larghezza desiderata;
   - il secondo argomento e' il numero da stampare. */
allinea(Larghezza, Numero) :- cifre(Numero, C),
                              Spazi is Larghezza - C,
                              stampa_spazi(Spazi),
                              write(Numero).

/* Il predicato cifre calcola il numero di cifre decimali di un numero
   naturale:
   - il primo argomento e' il numero di cui contare le cifre;
   - il secondo argomento e' il numero di cifre calcolato. */
cifre(Numero, 1) :- Numero < 10.
cifre(Numero, C) :- Numero >= 10,
                    Numero1 is Numero // 10,
                    cifre(Numero1, C1),
                    C is C1 + 1.

/* Il predicato stampa_spazi stampa un numero prefissato di spazi:
   - il suo unico argomento e' il numero di spazi da stampare. */
stampa_spazi(N) :- N =< 0.
stampa_spazi(N) :- N > 0, write(' '), N1 is N - 1, stampa_spazi(N1).

/* Il predicato nome_turno restituisce la descrizione testuale del modo
```

```
   di chiusura di un turno:
   - il primo argomento e' il turno da descrivere;
   - il secondo argomento e' la descrizione testuale. */
nome_turno(strike(_), 'strike').
nome_turno(spare(_), 'spare').
nome_turno(aperto(_), 'turno aperto').

/* Il predicato punteggio_totale calcola il punteggio complessivo della
   partita sommando i punteggi dei singoli turni:
   - il primo argomento e' la lista dei turni della partita;
   - il secondo argomento e' il punteggio totale calcolato. */
punteggio_totale([], 0).
punteggio_totale([Turno | Altri], Totale) :-
    punteggio(Turno, P),
    punteggio_totale(Altri, S),
    Totale is P + S.

/* Il predicato punteggio restituisce il punteggio di un turno,
   indipendentemente dal modo di chiusura del turno:
   - il primo argomento e' il turno di cui calcolare il punteggio;
   - il secondo argomento e' il punteggio del turno. */
punteggio(strike(P), P).
punteggio(spare(P), P).
punteggio(aperto(P), P).
```

5 Testing del Programma

Test Haskell 1 — Partita perfetta: dodici strike

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[10,10,10,10,10,10,10,10,10,10,10,10]

>> Punteggio dei singoli turni:
Turno  1:  punteggio 30   totale parziale  30   (strike)
Turno  2:  punteggio 30   totale parziale  60   (strike)
Turno  3:  punteggio 30   totale parziale  90   (strike)
Turno  4:  punteggio 30   totale parziale 120   (strike)
Turno  5:  punteggio 30   totale parziale 150   (strike)
Turno  6:  punteggio 30   totale parziale 180   (strike)
Turno  7:  punteggio 30   totale parziale 210   (strike)
Turno  8:  punteggio 30   totale parziale 240   (strike)
Turno  9:  punteggio 30   totale parziale 270   (strike)
Turno 10:  punteggio 30   totale parziale 300   (strike)
>> Punteggio totale della partita:
300
```

Test Haskell 2 — Partita interamente a vuoto

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]

>> Punteggio dei singoli turni:
Turno  1:  punteggio  0   totale parziale   0   (turno aperto)
Turno  2:  punteggio  0   totale parziale   0   (turno aperto)
Turno  3:  punteggio  0   totale parziale   0   (turno aperto)
Turno  4:  punteggio  0   totale parziale   0   (turno aperto)
Turno  5:  punteggio  0   totale parziale   0   (turno aperto)
Turno  6:  punteggio  0   totale parziale   0   (turno aperto)
Turno  7:  punteggio  0   totale parziale   0   (turno aperto)
Turno  8:  punteggio  0   totale parziale   0   (turno aperto)
Turno  9:  punteggio  0   totale parziale   0   (turno aperto)
Turno 10:  punteggio  0   totale parziale   0   (turno aperto)
>> Punteggio totale della partita:
0
```

Test Haskell 3 — Partita mista

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[1,4,4,5,6,4,5,5,10,0,1,7,3,6,4,10,2,8,6]

>> Punteggio dei singoli turni:
Turno  1:  punteggio  5   totale parziale   5   (turno aperto)
Turno  2:  punteggio  9   totale parziale  14   (turno aperto)
Turno  3:  punteggio 15   totale parziale  29   (spare)
Turno  4:  punteggio 20   totale parziale  49   (spare)
```

```
Turno  5:  punteggio 11   totale parziale  60   (strike)
Turno  6:  punteggio  1   totale parziale  61   (turno aperto)
Turno  7:  punteggio 16   totale parziale  77   (spare)
Turno  8:  punteggio 20   totale parziale  97   (spare)
Turno  9:  punteggio 20   totale parziale 117   (strike)
Turno 10:  punteggio 16   totale parziale 133   (spare)
>> Punteggio totale della partita:
133
```

Test Haskell 4 — Partita di esempio dell'analisi del problema

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[10,10,10,7,2,8,2,0,9,10,7,3,9,0,10,10,8]

>> Punteggio dei singoli turni:
Turno  1:  punteggio 30   totale parziale  30   (strike)
Turno  2:  punteggio 27   totale parziale  57   (strike)
Turno  3:  punteggio 19   totale parziale  76   (strike)
Turno  4:  punteggio  9   totale parziale  85   (turno aperto)
Turno  5:  punteggio 10   totale parziale  95   (spare)
Turno  6:  punteggio  9   totale parziale 104   (turno aperto)
Turno  7:  punteggio 20   totale parziale 124   (strike)
Turno  8:  punteggio 19   totale parziale 143   (spare)
Turno  9:  punteggio  9   totale parziale 152   (turno aperto)
Turno 10:  punteggio 28   totale parziale 180   (strike)
>> Punteggio totale della partita:
180
```

Test Haskell 5 — Dieci spare consecutivi

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5]

>> Punteggio dei singoli turni:
Turno  1:  punteggio 15   totale parziale  15   (spare)
Turno  2:  punteggio 15   totale parziale  30   (spare)
Turno  3:  punteggio 15   totale parziale  45   (spare)
Turno  4:  punteggio 15   totale parziale  60   (spare)
Turno  5:  punteggio 15   totale parziale  75   (spare)
Turno  6:  punteggio 15   totale parziale  90   (spare)
Turno  7:  punteggio 15   totale parziale 105   (spare)
Turno  8:  punteggio 15   totale parziale 120   (spare)
Turno  9:  punteggio 15   totale parziale 135   (spare)
Turno 10:  punteggio 15   totale parziale 150   (spare)
>> Punteggio totale della partita:
150
```

Test Haskell 6 — Nove strike e decimo turno aperto

```
ghci> main
```

```
Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[10,10,10,10,10,10,10,10,10,4,5]

>> Punteggio dei singoli turni:
Turno  1:  punteggio 30   totale parziale  30   (strike)
Turno  2:  punteggio 30   totale parziale  60   (strike)
Turno  3:  punteggio 30   totale parziale  90   (strike)
Turno  4:  punteggio 30   totale parziale 120   (strike)
Turno  5:  punteggio 30   totale parziale 150   (strike)
Turno  6:  punteggio 30   totale parziale 180   (strike)
Turno  7:  punteggio 30   totale parziale 210   (strike)
Turno  8:  punteggio 24   totale parziale 234   (strike)
Turno  9:  punteggio 19   totale parziale 253   (strike)
Turno 10:  punteggio  9   totale parziale 262   (turno aperto)
>> Punteggio totale della partita:
262
```

Test Haskell 7 — Spare nel decimo turno con lancio di bonus a strike

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[10,9,1,5,5,7,2,10,10,10,9,0,8,2,9,1,10]

>> Punteggio dei singoli turni:
Turno  1:  punteggio 20   totale parziale  20   (strike)
Turno  2:  punteggio 15   totale parziale  35   (spare)
Turno  3:  punteggio 17   totale parziale  52   (spare)
Turno  4:  punteggio  9   totale parziale  61   (turno aperto)
Turno  5:  punteggio 30   totale parziale  91   (strike)
Turno  6:  punteggio 29   totale parziale 120   (strike)
Turno  7:  punteggio 19   totale parziale 139   (strike)
Turno  8:  punteggio  9   totale parziale 148   (turno aperto)
Turno  9:  punteggio 19   totale parziale 167   (spare)
Turno 10:  punteggio 20   totale parziale 187   (spare)
>> Punteggio totale della partita:
187
```

Test Haskell 8 — Sequenza incompleta, poi corretta

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
>> Partita non valida.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]

>> Punteggio dei singoli turni:
Turno  1:  punteggio  0   totale parziale   0   (turno aperto)
Turno  2:  punteggio  0   totale parziale   0   (turno aperto)
Turno  3:  punteggio  0   totale parziale   0   (turno aperto)
Turno  4:  punteggio  0   totale parziale   0   (turno aperto)
Turno  5:  punteggio  0   totale parziale   0   (turno aperto)
Turno  6:  punteggio  0   totale parziale   0   (turno aperto)
Turno  7:  punteggio  0   totale parziale   0   (turno aperto)
```

```
Turno  8:  punteggio  0   totale parziale   0   (turno aperto)
Turno  9:  punteggio  0   totale parziale   0   (turno aperto)
Turno 10:  punteggio  0   totale parziale   0   (turno aperto)
>> Punteggio totale della partita:
0
```

Test Haskell 9 — Coppia di lanci non ammissibile, poi corretta

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[7,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
>> Partita non valida.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[7,3,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]

>> Punteggio dei singoli turni:
Turno  1:  punteggio 10   totale parziale  10   (spare)
Turno  2:  punteggio  0   totale parziale  10   (turno aperto)
Turno  3:  punteggio  0   totale parziale  10   (turno aperto)
Turno  4:  punteggio  0   totale parziale  10   (turno aperto)
Turno  5:  punteggio  0   totale parziale  10   (turno aperto)
Turno  6:  punteggio  0   totale parziale  10   (turno aperto)
Turno  7:  punteggio  0   totale parziale  10   (turno aperto)
Turno  8:  punteggio  0   totale parziale  10   (turno aperto)
Turno  9:  punteggio  0   totale parziale  10   (turno aperto)
Turno 10:  punteggio  0   totale parziale  10   (turno aperto)
>> Punteggio totale della partita:
10
```

Test Haskell 10 — Lanci in eccesso, poi corretti

```
ghci> main

Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5]
>> Partita non valida.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre:
[5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5]

>> Punteggio dei singoli turni:
Turno  1:  punteggio 15   totale parziale  15   (spare)
Turno  2:  punteggio 15   totale parziale  30   (spare)
Turno  3:  punteggio 15   totale parziale  45   (spare)
Turno  4:  punteggio 15   totale parziale  60   (spare)
Turno  5:  punteggio 15   totale parziale  75   (spare)
Turno  6:  punteggio 15   totale parziale  90   (spare)
Turno  7:  punteggio 15   totale parziale 105   (spare)
Turno  8:  punteggio 15   totale parziale 120   (spare)
Turno  9:  punteggio 15   totale parziale 135   (spare)
Turno 10:  punteggio 15   totale parziale 150   (spare)
>> Punteggio totale della partita:
150
```

Test Prolog 1 — Partita perfetta: dodici strike

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[10,10,10,10,10,10,10,10,10,10,10,10].

>> Punteggio dei singoli turni:
Turno  1:  punteggio 30   totale parziale  30   (strike)
Turno  2:  punteggio 30   totale parziale  60   (strike)
Turno  3:  punteggio 30   totale parziale  90   (strike)
Turno  4:  punteggio 30   totale parziale 120   (strike)
Turno  5:  punteggio 30   totale parziale 150   (strike)
Turno  6:  punteggio 30   totale parziale 180   (strike)
Turno  7:  punteggio 30   totale parziale 210   (strike)
Turno  8:  punteggio 30   totale parziale 240   (strike)
Turno  9:  punteggio 30   totale parziale 270   (strike)
Turno 10:  punteggio 30   totale parziale 300   (strike)
>> Punteggio totale della partita:
300

true ?

(2 ms) yes
```

Test Prolog 2 — Partita interamente a vuoto

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0].

>> Punteggio dei singoli turni:
Turno  1:  punteggio  0   totale parziale   0   (turno aperto)
Turno  2:  punteggio  0   totale parziale   0   (turno aperto)
Turno  3:  punteggio  0   totale parziale   0   (turno aperto)
Turno  4:  punteggio  0   totale parziale   0   (turno aperto)
Turno  5:  punteggio  0   totale parziale   0   (turno aperto)
Turno  6:  punteggio  0   totale parziale   0   (turno aperto)
Turno  7:  punteggio  0   totale parziale   0   (turno aperto)
Turno  8:  punteggio  0   totale parziale   0   (turno aperto)
Turno  9:  punteggio  0   totale parziale   0   (turno aperto)
Turno 10:  punteggio  0   totale parziale   0   (turno aperto)
>> Punteggio totale della partita:
0

true ?

yes
```

Test Prolog 3 — Partita mista

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[1,4,4,5,6,4,5,5,10,0,1,7,3,6,4,10,2,8,6].

>> Punteggio dei singoli turni:
Turno  1:  punteggio  5   totale parziale   5   (turno aperto)
```

```
Turno  2:  punteggio  9   totale parziale  14   (turno aperto)
Turno  3:  punteggio 15   totale parziale  29   (spare)
Turno  4:  punteggio 20   totale parziale  49   (spare)
Turno  5:  punteggio 11   totale parziale  60   (strike)
Turno  6:  punteggio  1   totale parziale  61   (turno aperto)
Turno  7:  punteggio 16   totale parziale  77   (spare)
Turno  8:  punteggio 20   totale parziale  97   (spare)
Turno  9:  punteggio 20   totale parziale 117   (strike)
Turno 10:  punteggio 16   totale parziale 133   (spare)
>> Punteggio totale della partita:
133

true ?

yes
```

Test Prolog 4 — Partita di esempio dell'analisi del problema

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[10,10,10,7,2,8,2,0,9,10,7,3,9,0,10,10,8].

>> Punteggio dei singoli turni:
Turno  1:  punteggio 30   totale parziale  30   (strike)
Turno  2:  punteggio 27   totale parziale  57   (strike)
Turno  3:  punteggio 19   totale parziale  76   (strike)
Turno  4:  punteggio  9   totale parziale  85   (turno aperto)
Turno  5:  punteggio 10   totale parziale  95   (spare)
Turno  6:  punteggio  9   totale parziale 104   (turno aperto)
Turno  7:  punteggio 20   totale parziale 124   (strike)
Turno  8:  punteggio 19   totale parziale 143   (spare)
Turno  9:  punteggio  9   totale parziale 152   (turno aperto)
Turno 10:  punteggio 28   totale parziale 180   (strike)
>> Punteggio totale della partita:
180

true ?

(1 ms) yes
```

Test Prolog 5 — Dieci spare consecutivi

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5].

>> Punteggio dei singoli turni:
Turno  1:  punteggio 15   totale parziale  15   (spare)
Turno  2:  punteggio 15   totale parziale  30   (spare)
Turno  3:  punteggio 15   totale parziale  45   (spare)
Turno  4:  punteggio 15   totale parziale  60   (spare)
Turno  5:  punteggio 15   totale parziale  75   (spare)
Turno  6:  punteggio 15   totale parziale  90   (spare)
Turno  7:  punteggio 15   totale parziale 105   (spare)
Turno  8:  punteggio 15   totale parziale 120   (spare)
Turno  9:  punteggio 15   totale parziale 135   (spare)
```

```
Turno 10:  punteggio 15   totale parziale 150   (spare)
>> Punteggio totale della partita:
150

true ?

(1 ms) yes
```

Test Prolog 6 — Nove strike e decimo turno aperto

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[10,10,10,10,10,10,10,10,10,4,5].

>> Punteggio dei singoli turni:
Turno  1:  punteggio 30   totale parziale  30   (strike)
Turno  2:  punteggio 30   totale parziale  60   (strike)
Turno  3:  punteggio 30   totale parziale  90   (strike)
Turno  4:  punteggio 30   totale parziale 120   (strike)
Turno  5:  punteggio 30   totale parziale 150   (strike)
Turno  6:  punteggio 30   totale parziale 180   (strike)
Turno  7:  punteggio 30   totale parziale 210   (strike)
Turno  8:  punteggio 24   totale parziale 234   (strike)
Turno  9:  punteggio 19   totale parziale 253   (strike)
Turno 10:  punteggio  9   totale parziale 262   (turno aperto)
>> Punteggio totale della partita:
262

true ?

yes
```

Test Prolog 7 — Spare nel decimo turno con lancio di bonus a strike

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[10,9,1,5,5,7,2,10,10,10,9,0,8,2,9,1,10].

>> Punteggio dei singoli turni:
Turno  1:  punteggio 20   totale parziale  20   (strike)
Turno  2:  punteggio 15   totale parziale  35   (spare)
Turno  3:  punteggio 17   totale parziale  52   (spare)
Turno  4:  punteggio  9   totale parziale  61   (turno aperto)
Turno  5:  punteggio 30   totale parziale  91   (strike)
Turno  6:  punteggio 29   totale parziale 120   (strike)
Turno  7:  punteggio 19   totale parziale 139   (strike)
Turno  8:  punteggio  9   totale parziale 148   (turno aperto)
Turno  9:  punteggio 19   totale parziale 167   (spare)
Turno 10:  punteggio 20   totale parziale 187   (spare)
>> Punteggio totale della partita:
187

true ?

yes
```

Test Prolog 8 — Sequenza incompleta, poi corretta

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0].
>> Partita non valida.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0].

>> Punteggio dei singoli turni:
Turno  1:  punteggio  0   totale parziale   0   (turno aperto)
Turno  2:  punteggio  0   totale parziale   0   (turno aperto)
Turno  3:  punteggio  0   totale parziale   0   (turno aperto)
Turno  4:  punteggio  0   totale parziale   0   (turno aperto)
Turno  5:  punteggio  0   totale parziale   0   (turno aperto)
Turno  6:  punteggio  0   totale parziale   0   (turno aperto)
Turno  7:  punteggio  0   totale parziale   0   (turno aperto)
Turno  8:  punteggio  0   totale parziale   0   (turno aperto)
Turno  9:  punteggio  0   totale parziale   0   (turno aperto)
Turno 10:  punteggio  0   totale parziale   0   (turno aperto)
>> Punteggio totale della partita:
0

true ?

(1 ms) yes
```

Test Prolog 9 — Coppia di lanci non ammissibile, poi corretta

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[7,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0].
>> Partita non valida.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[7,3,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0].

>> Punteggio dei singoli turni:
Turno  1:  punteggio 10   totale parziale  10   (spare)
Turno  2:  punteggio  0   totale parziale  10   (turno aperto)
Turno  3:  punteggio  0   totale parziale  10   (turno aperto)
Turno  4:  punteggio  0   totale parziale  10   (turno aperto)
Turno  5:  punteggio  0   totale parziale  10   (turno aperto)
Turno  6:  punteggio  0   totale parziale  10   (turno aperto)
Turno  7:  punteggio  0   totale parziale  10   (turno aperto)
Turno  8:  punteggio  0   totale parziale  10   (turno aperto)
Turno  9:  punteggio  0   totale parziale  10   (turno aperto)
Turno 10:  punteggio  0   totale parziale  10   (turno aperto)
>> Punteggio totale della partita:
10

true ?

(1 ms) yes
```

Test Prolog 10 — Lanci in eccesso, poi corretti

```
| ?- main.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5].
>> Partita non valida.
Digita i lanci separati da virgole e racchiusi tra parentesi quadre, terminando con un punto:
[5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5].

>> Punteggio dei singoli turni:
Turno  1:  punteggio 15   totale parziale  15   (spare)
Turno  2:  punteggio 15   totale parziale  30   (spare)
Turno  3:  punteggio 15   totale parziale  45   (spare)
Turno  4:  punteggio 15   totale parziale  60   (spare)
Turno  5:  punteggio 15   totale parziale  75   (spare)
Turno  6:  punteggio 15   totale parziale  90   (spare)
Turno  7:  punteggio 15   totale parziale 105   (spare)
Turno  8:  punteggio 15   totale parziale 120   (spare)
Turno  9:  punteggio 15   totale parziale 135   (spare)
Turno 10:  punteggio 15   totale parziale 150   (spare)
>> Punteggio totale della partita:
150

true ?

(2 ms) yes
```